

My Library Application in App Inventor 2

Here's another tutorial in our ongoing series to help you hone your skills for making useful Android apps with App Inventor.



It's a good practice to keep things organised and in place. This library app will help you to arrange your books and to find any book quickly when needed. The app pretty much resembles a real life library, where all the publications are arranged in definite categories for readers to have easy access to them.

So let's continue on our journey to master App Inventor. We have already had around eight to nine sessions, enough to make you comfortable with the capabilities of App Inventor. If you are reading this tutorial series for the first time, I would recommend that you familiarise yourself with the previous articles as well. That will help you understand the things we are going to do in this article, better.

Let's now proceed to build an Android app that will serve as a digital book library for us. Let's add new books to it and when needed, we will search for a required book and access more details that we have already saved. For the first time, I am introducing a list picker and the associated list block. After this tutorial you will learn to:

- Create a list
- Add items to the list
- Remove list items
- View all list items
- Search the list and use an index
- Join text to create a list item
- Split a list item
- Store lists in a database (Tiny DB)

- Retrieve data from the database
- Use procedures that return data
- Perform data validation

The purpose of the application

The application prepares a list of items that we have, and later refers to it to see what we have so far. The app is for a book library, which will enable us to add information about new books, edit the earlier entries, completely delete the records, etc. The app will be ideal in a 'paperless' real-life scenario, enabling you to write things digitally on to your mobile. You are already familiar with all the components that I will be using for this application, like buttons, labels, Tiny dB, horizontal arrangements, list pickers, etc.

GUI requirements

For every application, we have a graphical user interface or GUI, which helps us interact with the on-screen components. How each component responds to user actions is defined in the block editor section. As per our current requirements, we hope to have multiple text boxes and buttons with which you can write data and initiate methods.

The GUI requirements are:

Label: Labels are static text components, which are used to display some headings or markings on the screen.

Buttons: These let you trigger the event and are very essential components.